

可以把从 Linear Attention 到 Delta Attention 的演化理解成：

Linear Attention 学会了把无限增长的 KV Cache 压缩成固定状态 S ，但只会不断往 S 里加东西；Delta Attention 在此基础上增加“先读旧值、再写差值”的能力，使固定状态可以近似覆盖旧信息。下面统一采用：

$$q_t, k_t \in \mathbb{R}^{d_k}, \quad v_t, o_t \in \mathbb{R}^{d_v}$$

并把向量都看成行向量：

$$S_t \in \mathbb{R}^{d_k \times d_v}$$

于是：

$$k_t^\top v_t$$

是一个 $d_k \times d_v$ 的外积矩阵，而：

$$q_t S_t$$

得到 d_v 维输出。不同论文可能把 S 转置成 $d_v \times d_k$ ，写成 $S_t q_t$ ，本质完全一样。

一、从标准 Attention 开始

标准因果 Attention 在位置 t 的输出为：

$$o_t = \sum_{i=1}^t \frac{\exp(q_t k_i^\top)}{\sum_{j=1}^t \exp(q_t k_j^\top)} v_i$$

可以拆成三个步骤。

1. 计算 query 与历史 key 的相似度

$$s_{ti} = q_t k_i^\top$$

2. Softmax 归一化

$$a_{ti} = \frac{\exp(s_{ti})}{\sum_{j=1}^t \exp(s_{tj})}$$

3. 对 value 加权求和

$$o_t = \sum_{i=1}^t a_{ti} v_i$$

对应简化代码：

```
scores = q @ k.transpose(-1, -2)
scores = scores / math.sqrt(d)
scores = scores.masked_fill(causal_mask, float("-inf"))

attn = torch.softmax(scores, dim=-1)
out = attn @ v
```

对于 decode，虽然当前只有一个 query：

```
q.shape == [B, H, 1, D]
```

但仍需要读取全部历史 K/V：

```
k_cache.shape == [B, H, T, D]
v_cache.shape == [B, H, T, D]
```

单步 decode 的注意力读取量随历史长度 T 增长。

二、Linear Attention：把历史 K/V 压缩为固定状态

Linear Attention 不再使用：

$$\exp(q_t k_i^\top)$$

而是把相似度定义为可分解形式：

$$\text{sim}(q_t, k_i) = \phi(q_t) \phi(k_i)^\top$$

其中 ϕ 是非负特征映射，例如：

$$\phi(x) = \text{ELU}(x) + 1$$

于是输出变成：

$$o_t = \frac{\sum_{i=1}^t \phi(q_t) \phi(k_i)^\top v_i}{\sum_{i=1}^t \phi(q_t) \phi(k_i)^\top}$$

由于 $\phi(q_t)$ 与求和下标 i 无关，可以提到求和外面：

$$o_t = \frac{\phi(q_t) \left(\sum_{i=1}^t \phi(k_i)^\top v_i \right)}{\phi(q_t) \left(\sum_{i=1}^t \phi(k_i)^\top \right)}$$

定义两个状态：

$$S_t = \sum_{i=1}^t \phi(k_i)^\top v_i$$

$$z_t = \sum_{i=1}^t \phi(k_i)$$

则：

$$o_t = \frac{\phi(q_t) S_t}{\phi(q_t) z_t^\top}$$

状态递推为：

$$S_t = S_{t-1} + \phi(k_t)^\top v_t$$

$$z_t = z_{t-1} + \phi(k_t)$$

Linear Attention 不再保存长度为 T 的 K/V，而只保存：

$$S_t \in \mathbb{R}^{d_k \times d_v}$$

和：

$$z_t \in \mathbb{R}^{d_k}$$

状态大小不随序列长度增长。

三、Linear Attention 的代码

先看最容易理解的逐 token 参考实现。输入形状：

```
q: [B, H, T, Dk]
k: [B, H, T, Dk]
v: [B, H, T, Dv]
```

代码如下：

```
import torch
import torch.nn.functional as F

def feature_map(x):
    return F.elu(x) + 1.0

def linear_attention_recurrent(q, k, v, state=None, z_state=None):
    """
    q: [B, H, T, Dk]
    k: [B, H, T, Dk]
    v: [B, H, T, Dv]

    state: [B, H, Dk, Dv]
    z_state: [B, H, Dk]
    """

    q = feature_map(q)
```

```

k = feature_map(k)

B, H, T, Dk = q.shape
Dv = v.shape[-1]

if state is None:
    state = torch.zeros(
        B, H, Dk, Dv,
        device=q.device,
        dtype=q.dtype,
    )

if z_state is None:
    z_state = torch.zeros(
        B, H, Dk,
        device=q.device,
        dtype=q.dtype,
    )

outputs = []

for t in range(T):
    q_t = q[:, :, t] # [B, H, Dk]
    k_t = k[:, :, t] # [B, H, Dk]
    v_t = v[:, :, t] # [B, H, Dv]

    #  $k_t^T v_t$ 
    # [B,H,Dk,1] @ [B,H,1,Dv]
    # -> [B,H,Dk,Dv]
    kv_outer = (
        k_t.unsqueeze(-1)
        @ v_t.unsqueeze(-2)
    )

    # 更新固定状态
    state = state + kv_outer
    z_state = z_state + k_t

    #  $q_t S_t$ 
    # [B,H,1,Dk] @ [B,H,Dk,Dv]
    # -> [B,H,Dv]
    numerator = (

```

```

        q_t.unsqueeze(-2) @ state
    ).squeeze(-2)

    # q_t z_t
    denominator = (
        q_t * z_state
    ).sum(dim=-1, keepdim=True)

    o_t = numerator / denominator.clamp_min(1e-6)
    outputs.append(o_t)

output = torch.stack(outputs, dim=2)

return output, state, z_state

```

其中最核心的三行是：

```

state = state + k_t.unsqueeze(-1) @ v_t.unsqueeze(-2)
z_state = z_state + k_t
o_t = (q_t.unsqueeze(-2) @ state).squeeze(-2)

```

它们分别对应：

$$S_t = S_{t-1} + k_t^\top v_t$$

$$z_t = z_{t-1} + k_t$$

$$o_t = \frac{q_t S_t}{q_t z_t^\top}$$

这里为简洁省略了 ϕ 标记，因为代码已经提前执行了：

```

q = feature_map(q)
k = feature_map(k)

```

四、Linear Attention 为什么会越写越乱？

Linear Attention 的状态是：

$$S_t = k_1^\top v_1 + k_2^\top v_2 + \cdots + k_t^\top v_t$$

用某个 query q 读取：

$$qS_t = \sum_{i=1}^t (qk_i^\top) v_i$$

如果不同 key 完全正交：

$$k_i k_j^\top = 0, \quad i \neq j$$

那么用 k_i 读取时，只会取回 v_i 。但真实模型中的 key 通常不是完全正交的。例如：

$$k_1 = [1 \quad 0]$$

$$k_2 = [0.8 \quad 0.6]$$

它们的相似度为：

$$k_1 k_2^\top = 0.8$$

因此，用 k_1 读取状态时，不仅会读到 v_1 ，还会读到一部分 v_2 ：

$$k_1 S = v_1 + 0.8v_2 + \cdots$$

序列越长，更多 key-value 外积被加进同一个状态，干扰就可能越来越严重。

五、Linear Attention 直接重复写入的问题

假设状态里已经有：

$$k^\top v_{\text{old}}$$

现在相同或者相近的 key 又写入一个新值：

$$v_{\text{new}}$$

Linear Attention 直接执行：

$$S' = S + k^\top v_{\text{new}}$$

如果 k 是单位向量：

$$kk^\top = 1$$

用 k 读取新状态：

$$\begin{aligned} kS' &= kS + kk^\top v_{\text{new}} \\ &= v_{\text{old}} + v_{\text{new}} \end{aligned}$$

得到的是：

$$v_{\text{old}} + v_{\text{new}}$$

而不是：

$$v_{\text{new}}$$

所以 Linear Attention 只会叠加，不会覆盖。

六、Delta Attention：先读旧值，再写差值

Delta Rule 的核心公式是：

$$v_t^{\text{old}} = k_t S_{t-1}$$

先查看当前 key 方向上已经存了什么。然后计算旧值与目标值之间的差：

$$\Delta v_t = v_t - v_t^{\text{old}}$$

引入写入门：

$$u_t = \beta_t (v_t - k_t S_{t-1})$$

其中：

$$0 \leq \beta_t \leq 1$$

最后把差值写入状态：

$$S_t = S_{t-1} + k_t^\top u_t$$

即：

$$S_t = S_{t-1} + \beta_t k_t^\top (v_t - k_t S_{t-1})$$

输出通常写为：

$$o_t = q_t S_t$$

完整过程就是：

```

用 k_t 读取旧内容
    ↓
old = k_t S

计算距离目标 v_t 还差多少
    ↓
delta = v_t - old

根据 β_t 决定写入多少
    ↓
update = β_t × delta

沿 k_t 方向把差值写回状态
    ↓
S = S + k_t^T update

用 q_t 读取更新后的状态
    ↓
o_t = q_t S

```

七、为什么 Delta Attention 通常归一化 Q 和 K?

DeltaNet 中经常使用：

$$q_t = \text{normalize}(\text{SiLU}(W_q x_t))$$

$$k_t = \text{normalize}(\text{SiLU}(W_k x_t))$$

使得：

$$\|k_t\|_2 = 1$$

也就是：

$$k_t k_t^\top = 1$$

这时 Delta Rule 才能具有清晰的“覆盖”性质。假设更新前：

$$k_t S_{t-1} = v_{\text{old}}$$

更新后：

$$S_t = S_{t-1} + \beta_t k_t^\top (v_t - v_{\text{old}})$$

再次用 k_t 读取：

$$\begin{aligned} k_t S_t &= k_t S_{t-1} + \beta_t k_t k_t^\top (v_t - v_{\text{old}}) \\ &= v_{\text{old}} + \beta_t (v_t - v_{\text{old}}) \end{aligned}$$

即：

$$\boxed{k_t S_t = (1 - \beta_t) v_{\text{old}} + \beta_t v_t}$$

因此：

- $\beta_t = 0$ ：不更新；
- $\beta_t = 0.5$ ：旧值与目标值各占一半；
- $\beta_t = 1$ ：完全更新为目标值。当 $\beta_t = 1$ ：

$$k_t S_t = v_t$$

这就是 Delta Rule 的覆盖效果。

八、Delta Attention 的逐 token 代码

下面是与公式——对应的参考实现：

```
import torch
import torch.nn.functional as F

def delta_attention_recurrent(q, k, v, beta, state=None):
    """
    q:      [B, H, T, Dk]
    k:      [B, H, T, Dk]
    v:      [B, H, T, Dv]
    beta:   [B, H, T, 1]

    state:  [B, H, Dk, Dv]
    """

    # DeltaNet 常见处理
    q = F.normalize(F.silu(q), p=2, dim=-1)
    k = F.normalize(F.silu(k), p=2, dim=-1)

    B, H, T, Dk = q.shape
    Dv = v.shape[-1]

    if state is None:
        state = torch.zeros(
            B, H, Dk, Dv,
            device=q.device,
            dtype=q.dtype,
        )

    outputs = []

    for t in range(T):
        q_t = q[:, :, t]      # [B,H,Dk]
        k_t = k[:, :, t]      # [B,H,Dk]
        v_t = v[:, :, t]      # [B,H,Dv]
        beta_t = beta[:, :, t] # [B,H,1]

        # 1. old = k_t S_{t-1}
```

```

old_value = (
    k_t.unsqueeze(-2) @ state
).squeeze(-2)
# [B,H,Dv]

# 2.  $\delta = \beta_t * (v_t - old)$ 
delta = beta_t * (v_t - old_value)
# [B,H,Dv]

# 3.  $S_t = S_{t-1} + k_t^T \delta$ 
state = state + (
    k_t.unsqueeze(-1)
    @ delta.unsqueeze(-2)
)
# [B,H,Dk,Dv]

# 4.  $o_t = q_t S_t$ 
o_t = (
    q_t.unsqueeze(-2) @ state
).squeeze(-2)
# [B,H,Dv]

outputs.append(o_t)

output = torch.stack(outputs, dim=2)

return output, state

```

四段代码与公式完全对应：

```

old_value = k_t @ state

```

对应：

$$v_t^{\text{old}} = k_t S_{t-1}$$

```

delta = beta_t * (v_t - old_value)

```

对应：

$$u_t = \beta_t(v_t - k_t S_{t-1})$$



```
state = state + k_t.T @ delta
```

对应：

$$S_t = S_{t-1} + k_t^\top u_t$$



```
o_t = q_t @ state
```

对应：

$$o_t = q_t S_t$$

九、Linear 与 Delta 的代码差异

两者的主要差异实际上只有“写入内容”不同。

Linear Attention



```
update = k_t.unsqueeze(-1) @ v_t.unsqueeze(-2)
state = state + update
```

写入的是完整目标值：

$$k_t^\top v_t$$

Delta Attention



```
old_value = (
    k_t.unsqueeze(-2) @ state
```

```

).squeeze(-2)

delta = beta_t * (v_t - old_value)

update = (
    k_t.unsqueeze(-1)
    @ delta.unsqueeze(-2)
)

state = state + update

```

写入的是：

$$\beta_t k_t^\top (v_t - k_t S_{t-1})$$

可以总结为：

```

# Linear Attention
write_value = v_t

# Delta Attention
write_value = beta_t * (v_t - k_t @ state)

```

其余外积写入形式完全相同。

十、用具体数字走一遍代码

设：

$$d_k = d_v = 2$$

当前状态为：

$$S_{t-1} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

当前 key：

$$k_t = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

目标 value:

$$v_t = \begin{bmatrix} 0 & 1 \end{bmatrix}$$

写入强度:

$$\beta_t = 1$$

第一步：读取旧值

$$\begin{aligned} v_t^{\text{old}} &= k_t S_{t-1} \\ &= \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \end{bmatrix} \end{aligned}$$

代码:

```
old_value = k_t @ state
# [1, 0]
```

第二步：计算差值

$$\begin{aligned} u_t &= v_t - v_t^{\text{old}} \\ &= \begin{bmatrix} 0 & 1 \end{bmatrix} - \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} -1 & 1 \end{bmatrix} \end{aligned}$$

代码:

```
delta = v_t - old_value
# [-1, 1]
```

第三步：外积写入

$$\begin{aligned} k_t^\top u_t &= \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} \\ &= \begin{bmatrix} -1 & 1 \\ 0 & 0 \end{bmatrix} \end{aligned}$$

代码:

```
update = k_t.T @ delta
```

第四步：更新状态

$$\begin{aligned} S_t &= \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} -1 & 1 \\ 0 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \end{aligned}$$

代码:

```
state = state + update
```

第五步：重新读取

$$k_t S_t = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 \end{bmatrix}$$

正好等于目标值 v_t 。

十一、Delta Rule 的矩阵变形

从:

$$S_t = S_{t-1} + \beta_t k_t^\top (v_t - k_t S_{t-1})$$

展开:

$$S_t = S_{t-1} + \beta_t k_t^\top v_t - \beta_t k_t^\top k_t S_{t-1}$$

整理：

$$S_t = (I - \beta_t k_t^\top k_t) S_{t-1} + \beta_t k_t^\top v_t$$

这里：

$$I - \beta_t k_t^\top k_t$$

是状态转移矩阵。它可以理解成：

1. 沿 k_t 方向擦除一部分旧状态；
2. 再沿 k_t 方向写入新值。也就是：

$$\underbrace{(I - \beta_t k_t^\top k_t) S_{t-1}}_{\text{删除旧关联}} + \underbrace{\beta_t k_t^\top v_t}_{\text{写入新关联}}$$

当 $\beta_t = 1$ 时：

$$I - k_t^\top k_t$$

会把状态在 k_t 方向上的分量投影掉，然后写入新的 v_t 。这也是 DeltaNet 能被解释成 Fast Weight Programmer 和广义 Householder 状态转移的原因。

十二、为什么 Delta Attention 通常不再需要 z_t ？

原始 Linear Attention 使用：

$$o_t = \frac{q_t S_t}{q_t z_t^\top}$$

其中 z_t 用来归一化注意力权重。DeltaNet 通常采用归一化后的 Q、K：

$$q_t = \frac{\text{SiLU}(\tilde{q}_t)}{\|\text{SiLU}(\tilde{q}_t)\|_2}$$

$$k_t = \frac{\text{SiLU}(\tilde{k}_t)}{\|\text{SiLU}(\tilde{k}_t)\|_2}$$

并通过：

- Q/K 归一化；
- 学习到的 β_t ；
- 输出归一化；
- 门控或短卷积； 共同控制数值尺度，因此常直接使用：

$$o_t = q_t S_t$$

而不再维护：

$$z_t = \sum_i k_i$$

但这并不是说两者数学上完全等价，而是 DeltaNet 采用了另一套归一化与稳定化机制。

十三、完整的 Delta Attention 层

更接近真实模型的结构通常是：

```
class DeltaAttention(torch.nn.Module):
    def __init__(self, hidden_size, num_heads):
        super().__init__()

        self.hidden_size = hidden_size
        self.num_heads = num_heads
        self.head_dim = hidden_size // num_heads

        self.q_proj = torch.nn.Linear(hidden_size, hidden_size)
        self.k_proj = torch.nn.Linear(hidden_size, hidden_size)
        self.v_proj = torch.nn.Linear(hidden_size, hidden_size)

        self.beta_proj = torch.nn.Linear(hidden_size, num_heads)
        self.o_proj = torch.nn.Linear(hidden_size, hidden_size)

    def forward(self, x, state=None):
        B, T, C = x.shape
        H = self.num_heads
        D = self.head_dim
```

```

q = self.q_proj(x)
k = self.k_proj(x)
v = self.v_proj(x)

q = q.view(B, T, H, D).transpose(1, 2)
k = k.view(B, T, H, D).transpose(1, 2)
v = v.view(B, T, H, D).transpose(1, 2)

q = F.normalize(F.silu(q), dim=-1)
k = F.normalize(F.silu(k), dim=-1)

beta = torch.sigmoid(self.beta_proj(x))
beta = beta.transpose(1, 2).unsqueeze(-1)
# [B,H,T,1]

out, state = delta_attention_recurrent(
    q=q,
    k=k,
    v=v,
    beta=beta,
    state=state,
)

out = out.transpose(1, 2).reshape(B, T, C)
out = self.o_proj(out)

return out, state

```

真实实现通常还会加入：

- short convolution;
- RMSNorm;
- output gate;
- chunk-wise 并行算法;
- 自定义 Triton/CUDA kernel;
- FP32 recurrent state;
- 多种 head expansion;
- Gated DeltaNet 的状态衰减 α_t 。但核心递推仍是：

```
old = k @ state
delta = beta * (v - old)
state = state + k.T @ delta
out = q @ state
```

十四、Prefill 与 Decode 的实现差异

Decode

Decode 每次只有一个 token:

```
q.shape = [B, H, 1, D]
```

直接执行递归更新即可:

```
old = k @ state
delta = beta * (v - old)
state += k.transpose(-1, -2) @ delta
out = q @ state
```

每层只保存:

```
state.shape = [B, H, Dk, Dv]
```

状态大小不随上下文长度增长。

Prefill

Prefill 一次有很多 token:

```
q.shape = [B, H, T, D]
```

最直接的代码需要：

```
for t in range(T):
    ...
```

但这会形成长度为 T 的串行依赖，GPU 利用率很低。因此实际 DeltaNet 会把序列切成多个 chunk：

```
Chunk 1 → 最终状态  $S_1$ 
Chunk 2 → 最终状态  $S_2$ 
Chunk 3 → 最终状态  $S_3$ 
```

块内通过重参数化并行计算多个 token 的修正项，块间只传递最终状态。所以：

- decode 使用 recurrent kernel；
- prefill 使用 chunk-wise parallel kernel；
- 两者数学上实现的是同一个 Delta Rule。

十五、Linear Attention 与 Delta Attention 总结

项目	Linear Attention	Delta Attention
状态	$S_t = S_{t-1} + k_t^\top v_t$	$S_t = S_{t-1} + k_t^\top u_t$
写入内容	完整 v_t	差值 $u_t = \beta_t(v_t - k_t S_{t-1})$
是否读取旧状态	更新时不读取	更新前先读取
重复 key	新旧 value 相加	旧 value 被近似替换
写入门	通常没有	β_t
状态大小	固定	固定
长序列问题	信息不断叠加、发生干扰	可以修正和覆盖局部关联
主要局限	只加不删	key 不正交时仍有交叉干扰
Decode	固定状态递推	固定状态递推
Prefill	可分块并行	需通过重参数化分块并行

最终可以把两段代码压缩成下面这个对比：

```
# Linear Attention
state = state + outer(k, v)
out = q @ state

# Delta Attention
old = k @ state
delta = beta * (v - old)
state = state + outer(k, delta)
out = q @ state
```

最本质的变化就是：

Linear: 直接写 v_t

变成：

Delta: 写入 v_t — 当前已经读到的内容

因此，Delta Attention 并不是推翻 Linear Attention，而是在同一个固定状态框架中，把“纯加法内存”升级为“可读取、可修改、可覆盖的关联内存”。